

Reviewer Pack — Minimal Number of Real Points in Tropical Curves vs ACC⁰ Cir...

Entry 4b702ee28f81 · INCONCLUSIVE

SEC autonomous research engine — attribution: Ludovico Kubler

2026-05-22 11:27:26 UTC

Minimal Number of Real Points in Tropical Curves vs ACC⁰ Circuit Threshold

Entry ID: 4b702ee28f81 **Verdict:** INCONCLUSIVE **Recorded:** 2026-05-22 11:27:26 UTC

1. Conjecture

Field A (mathematical branch): Tropical Geometry **Field B** (complexity object): ACC⁰ Circuit Complexity

Statement:

{‘stmt1’: ‘Let f be a tropical polynomial on the interval $[0,1]$ with computable coefficients over the real numbers.’, ‘stmt2’: ‘The minimal number of real points in any curve defined by f is denoted as $\text{MRP}(f)$.’, ‘stmt3’: ‘Then for every explicit function in $\mathcal{P}$, if there exists an ACC⁰ circuit computing it with depth D and size S , then $\text{MRP}(f) \geq 2^{(D/2 + \epsilon S)}$ for some $\epsilon > 0$.’}

Rationale (proposer’s reasoning):

{‘stmt1’: ‘Tropical geometry provides a unique way to represent real numbers in a structured manner that is amenable to analysis.’, ‘stmt2’: ‘The minimal number of real points in a tropical curve is an invariant that can be computed efficiently for polynomials.’, ‘stmt3’: ‘This conjecture suggests a potential connection between the geometric properties of tropical curves and circuit complexity, which could lead to new insights into ACC⁰ circuits.’}

Taxonomy category: ACC_SIPSER (status at proposal time:)

2. Pre-registration (Popper-style)

Hash (SHA-256 prefix): 8c72b3371d2af757

This hash commits to the conjecture statement, acceptance criterion, and seed list **before** the empirical test runs. Tampering with the test or its data after this hash was computed would be detectable.

Acceptance criterion:

The conjecture is supported if, for at least 80% of seeds, $\text{MRP}(f) \geq 2^{(D/2 + \epsilon S)}$ with a mean difference between MRP and the threshold ≤ 1 .

3. Barrier filter (F1)

Final: PASS

Barrier	Verdict	Confidence	LLM ₁	LLM ₂
RELATIVIZATION	UNCERTAIN	1.00	SAFE	HITS
NATURAL_PROOFS	SAFE	0.50	UNCERTAIN	UNCERTAIN
ALGEBRIZATION	SAFE	0.50	UNCERTAIN	UNCERTAIN
KARP_LIPTON	SAFE	0.50	UNCERTAIN	UNCERTAIN

4. Novelty audit

Verdict: ADJACENT_OK against 5 hits across arXiv + Semantic Scholar + ECCC

Search queries (3): - tropical geometry AND ACC⁰ circuit complexity - minimal number of real points tropical curves AND ACC⁰ circuit threshold - computable coefficients over real numbers AND MRP(f) $\geq 2^{(D/2 + \epsilon S)}$

Top relevant hits considered: - [http://arxiv.org/abs/2603.25938v1] Narrowband searches for continuous gravitational waves from known pulsars in the first two parts of the fourth LIGO-Virgo - [http://arxiv.org/abs/1402.4974v3] Implementation of an F-statistic all-sky search for continuous gravitational waves in Virgo VSR1 data - [http://arxiv.org/abs/2112.06861v3] Tests of General Relativity with GWTC-3 - [s2:2502.10590] Optimal Bounds for the Number of Pieces of Near-Circuit Hypersurfaces - [s2:2403.17963] A better compression driver? CutFEM 3D shape optimization taking viscothermal losses into account

5. Empirical test harness

Multi-seed protocol: 5 seeds (11, 23, 37, 53, 71)

Sandbox: isolated subprocess, stdlib-only, ≤ 90 s wall time per cycle **Execution:** rc=1, elapsed=0.1s

5.1 Generated Python source

```
import random
import math
from fractions import Fraction

def run_trial(seed: int) -> dict:
    random.seed(seed)

    # Generate a random tropical polynomial f with coefficients over [0, 1]
    n = random.randint(5, 40)
    coeffs = [random.random() for _ in range(n + 1)]
    x = 0.5 # Evaluate the polynomial at x = 0.5
    f = sum(c * x**i for i, c in enumerate(coeffs))

    # Construct an ACC0 circuit for a simple function (e.g., identity)
    D = random.randint(2, 10)
    S = random.randint(5, 20)
    # Simulate the circuit evaluation
    result = f

    # Compute the minimal number of real points MRP(f) in the tropical curve defined by f
    MRP_f = len([i for i in range(n + 1) if coeffs[i] != 0])

    # Compare MRP(f) with the predicted threshold 2^(D/2 + εS)
    epsilon = 0.1
    threshold = 2**(D / 2 + epsilon * S)
```

```

conjecture_holds = MRP_f >= threshold
counterexample = "" if conjecture_holds else f"MRP(f)={MRP_f}, threshold={threshold}"

return {
    "metric_name": "MRP(f)",
    "metric_value": MRP_f,
    "instances_tested": 1,
    "conjecture_holds": conjecture_holds,
    "counterexample": counterexample
}

if __name__ == "__main__":
    seeds = [int(arg) for arg in sys.argv[1:]] or [random.getrandbits(32) for _ in range(30)]

    results = []
    for seed in seeds:
        trial_result = run_trial(seed)
        print(f"TRIAL: {trial_result}")
        results.append(trial_result)

    mean_value = sum(result["metric_value"] for result in results) / len(results)
    std_dev = math.sqrt(sum((result["metric_value"] - mean_value)**2 for result in results) / len(results))
    support_fraction = sum(result["conjecture_holds"] for result in results) / len(results)

    if support_fraction >= 0.8:
        print(f"RESULT: SUPPORTED mean={mean_value} std={std_dev} support_fraction={support_fraction}")
    elif any(not result["conjecture_holds"] for result in results):
        first_failing_seed = next(seed for seed, result in zip(seeds, results) if not result["conjecture_holds"])
        print(f"RESULT: FALSIFIED counterexample=\"{result['counterexample']}\" first_failing_seed={first_failing_seed}")
    else:
        print("RESULT: INCONCLUSIVE insufficient_data")

```

6. Per-seed results

(no seed-level data — test crashed before producing TRIAL: lines)

7. Test stdout (last 2KB)

```

59.71411145835569'}
TRIAL: {'metric_name': 'MRP(f)', 'metric_value': 19, 'instances_tested': 1, 'conjecture_holds': False}
TRIAL: {'metric_name': 'MRP(f)', 'metric_value': 16, 'instances_tested': 1, 'conjecture_holds': False}
TRIAL: {'metric_name': 'MRP(f)', 'metric_value': 8, 'instances_tested': 1, 'conjecture_holds': False}
TRIAL: {'metric_name': 'MRP(f)', 'metric_value': 26, 'instances_tested': 1, 'conjecture_holds': True}
TRIAL: {'metric_name': 'MRP(f)', 'metric_value': 24, 'instances_tested': 1, 'conjecture_holds': False}
TRIAL: {'metric_name': 'MRP(f)', 'metric_value': 10, 'instances_tested': 1, 'conjecture_holds': True}
TRIAL: {'metric_name': 'MRP(f)', 'metric_value': 39, 'instances_tested': 1, 'conjecture_holds': True}
TRIAL: {'metric_name': 'MRP(f)', 'metric_value': 17, 'instances_tested': 1, 'conjecture_holds': False}
TRIAL: {'metric_name': 'MRP(f)', 'metric_value': 37, 'instances_tested': 1, 'conjecture_holds': True}
TRIAL: {'metric_name': 'MRP(f)', 'metric_value': 20, 'instances_tested': 1, 'conjecture_holds': False}
TRIAL: {'metric_name': 'MRP(f)', 'metric_value': 18, 'instances_tested': 1, 'conjecture_holds': True}

--STDERR--
Traceback (most recent call last):
  File "/home/ludo/Scrivania/SEC/research/pvsnp_sandbox/test_29ce203f.py", line 68, in <module>
    print(f"RESULT: FALSIFIED counterexample=\"{result['counterexample']}\" first_failing_seed={first_failing_seed}")
    ~~~~~~

```

NameError: name 'result' is not defined. Did you mean: 'results'?

8. Critic adversarial review

Critic verdict: CHALLENGE

Critic reasoning:

skipped: test crashed before producing data

9. Final verdict & safety rail

Verdict: INCONCLUSIVE

Reasoning:

Safety rail: critic_challenge_falsified | original: The conjecture was falsified as there are instances where $\overline{\text{MRP}}(f) < \overline{\text{threshold}}$, which violates the conjecture's condition. | next: Investigate further to identify patterns in counterexamples and refine the conjecture or explore alternative explanations.

11. Audit log (LLM calls)

Total LLM calls: 9

#	Phase	Provider	Model	Tokens in	out	Latency (ms)
1	propose	ollama_remote	glm4:latest	0	0	14606
2	preregistration	ollama_remote	glm4:latest	0	0	9305
3	novelty	ollama_remote	glm4:latest	0	0	9019
4	novelty	ollama_remote	glm4:latest	0	0	9527
5	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	14457
6	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	13564
7	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	8488
8	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	7504
9	judge	ollama_remote	glm4:latest	0	0	11794

Totals: 0 input tokens, 0 output tokens, ~\$0.0000 cost, 98264 ms total latency. Provider mix: {'ollama_remote': 9}

(full prompt+response transcripts available in `research/audit/4b702ee28f81.jsonl`)

12. Reproducibility

To reproduce this cycle:

1. Apply the test harness in section 5.1 with seeds 11,23,37,53,71
2. The Python sandbox is pure-stdlib; any Python 3.11+ should suffice
3. The full LLM transcripts are in `research/audit/4b702ee28f81.jsonl` for verification of provenance
4. The pre-registration hash in section 2 commits to the test design before execution; tampering would be detectable.

Sandbox file: `research/pvsnp_sandbox/test_*.py` (per-cycle)

Replay tarball: `research/replay/4b702ee28f81.tar.gz` (if generated)